
DART: Demand-Addressed Runtime Tokens for Interest-Driven Decode

ARXIV PREPRINT

Aditya Raj
Independent Researcher

September 2026

ABSTRACT

Autoregressive LLM serving still treats decode as a push problem: the engine generates as fast as the GPU allows, then a byte pipe tries to deliver. A token with no consumer credit is a wasted residual-stream step—energy, KV pages, and batch slots spent encoding a segment nobody asked to absorb. Andes noticed that humans read slower than GPUs generate, then still generated and buffered. This paper inverts the stack.

Interest-Driven Decode (IDD) makes an outstanding Interest the only thing that may run a decode kernel. Zero Interests means zero decode. A cache hit never touches the kernel. Changing machines is not live-migration of an HTTP request; it is a different producer answering the next named Interest.

DART (Demand-Addressed Runtime Tokens) is the continuation runtime that implements IDD. After prefill it publishes a capability—model fingerprint, Merkle `kv_root`, anti-amplification lease—not a process snapshot. Consumers issue Interests for a token window W . The scheduler *is* the congestion controller: slow-start then AIMD over tokens, not bytes. Named Data is immutable and cacheable under the Continuation Interest Protocol (CIP). Routing is CAS, then the live pin holder, then the cheapest node plus KV handover and `adopt` without re-prefill. The kernel remains vLLM, llama.cpp, an in-process Hugging Face model, or a synthetic producer with real KV-extent accounting.

This is a control plane, not a GPU fleet, and the evaluation is honest about that. On the in-repository paper suite (`dart experiment -suite paper`, 1.0 s, 96-token cap, synthetic producer), a 30 tok/s reading pacer cuts generated tokens 52% and KV high-water 43% versus push, with zero unread inventory. Andes captures 68% of that generated-token win and still holds a 32-token watermark; the Andes-complete kill does not fire. A JSON burst pacer generates 21 tokens instead of 96. Grammar jump-forward is one kernel versus 36 under logit masking. A FileCAS peer answers with zero engine launches. A three-node in-process mesh pins, CAS-hits, prefers the expensive pin holder, then hands 512 KiB of named KV to the cheapest node without a new prefill. An in-process waiting plugin admits once and parks in `waiting` with pinned blocks; the HTTP vLLM adapter re-enters admission on every Interest.

DART does not replace vLLM. It decides whether vLLM may run.

Keywords. LLM serving, interest-driven decode, named continuations, congestion control, KV cache

1 Introduction

Today’s serving stacks maximize occupancy [1–4]. The scheduler packs sequences. The network is a pipe. The client is a sink. Prefill/decode disaggregation [5, 6] and KV-centric fabrics [7–9] move the residual stream; they do not ask whether it should run.

That question matters. Autoregression is the expensive encoder. If the compositor, TTS mouth, or JSON parser cannot absorb the next segment, encoding it is waste: joules in the residual stream, pages in the KV cache, and a batch slot that another paid request could have used. FlashAttention [10] and TPU serving [11] made the kernel cheaper. They did not change *whether* it may run.

Andes [12] is the closest prior system in spirit. It couples a display watermark to generation so a human reader is not flooded. The kernel still runs until the buffer hits the watermark. The inventory of unread tokens is the tell: Andes paid residual-stream steps for tokens the compositor has not absorbed. A pause is not a refuse.

Interest-Driven Decode inverts the default:

Outstanding Interests are the only thing that may run a decode kernel.

Zero Interests means zero decode. A named object already in a content-addressed store never touches the kernel. Disconnect (SSE cancel, WebSocket close, DELETE) zeroes credits. That is how backpressure reaches the kernel, which stock vLLM SSE does not: the engine keeps generating after the gateway stops reading.

DART (Demand-Addressed Runtime Tokens) is the runtime that makes that invariant true in front of an existing engine. A live generation is an address space, not a socket:

$$/cip/\langle h_{\text{model}} \rangle / \langle kv_root \rangle / \text{tokens} / \text{seg} / \langle i \rangle. \quad (1)$$

After prefill, DART publishes a capability, not a vLLM process snapshot: model fingerprint, Merkle `kv_root`, window cap, decode quota, expiry, startup credit. Consumers—compositor, TTS, JSON parser, tool runtime, another model—estimate a receive window and emit Interests. Pacers in the SDK turn those rates into W : `DrainPacer` (API pull), `ReadingPacer` (default 30 tok/s), `TtsPacer`, `JsonNeedPacer` (burst then pause).

The scheduler is a congestion controller over tokens [13]. Each continuation tracks a window `cwnd`, granted credits, in-flight tokens, a slow-start threshold, and an RTT estimate. The invariant is

$$\text{credits} + \text{in-flight} \leq \min(\text{cwnd}, W_{\text{max}}^{\text{lease}}, W_{\text{max}}). \quad (2)$$

Slow-start then AIMD on ACK (the next Interest). Timeout halves the window, pins KV, and discards speculative draft. Speculative K is exported as a TCP-style prefetch *signal*. v1 does not run uncredited residual-stream steps: that would violate IDD, and HTTP adapters cannot roll back a draft.

Named Data is the cache unit. `PinnedKVPool` indexes engine state by `kv_root`. Sleep does not drop identity; expiry does. `adopt` installs named KV and skips prefill. `InterestRouter` order: (1) CAS already has the name; (2) live pin holder, even if expensive; (3) live continuation; (4) cheapest node, NIXL-shaped pull, `adopt`, decode. Changing machines is answering the next Interest from a new locator.

DART never owns the residual stream. `SyntheticEngine` is the kill-test producer (hash-based tokens, real layer×block KV extents). `VLLMChatEngine` and `LlamaCppEngine` turn each credited Interest into `max_tokens=W / n_predict=W`. `HuggingFaceEngine` runs in-process weights (demo: `SmolLM2-135M-Instruct`). `InProcessVLLMEngine` admits once, then parks in waiting with pinned blocks when $W=0$. `CacheOnlyEngine` never decodes.

This paper is the system that is implemented, not a fleet we did not build. HTTP adapters already sit on the same CIP. They re-enter admission; that is a limitation, not a footnote. Evaluation uses the in-repo paper suite on a synthetic producer with real KV-extent accounting, plus mesh, CAS-peer, grammar, and waiting-plugin ablations. We do not report GPU-fleet tok/s, tok/J, or Time-to-First-Token on models we did not run in this suite.

Contributions.

1. IDD as a primitive: credit-gated decode on named continuations, with a formal invariant, kill criteria, and a protocol that makes the invariant checkable.
2. CIP: immutable names, HMAC leases, Nacks, and AIMD over tokens rather than TCP byte windows or `max_tokens` abort/restart.
3. Pin, connector, and mesh: CAS $\rightarrow$ pin holder $\rightarrow$ cheapest-plus-handover routing without re-prefill, with `kv_root` as identity.
4. An Andes-complete kill test, a grammar jump-versus-mask ablation, a FileCAS peer, a waiting-plugin versus HTTP admission comparison, and honest limits of the HTTP path.

What this is not. DART is not a new CUDA kernel, a new NDN stack, an IETF draft, or a replacement for vLLM. MOQT [14] is a future *delivery* profile, not the decode authorizer. SGLang [15] and XGrammar [16] jump structured spans inside one process; DART names the span so a peer can cache it.

Section 2 places IDD against occupancy serving, disaggregation, QoE pause, and named data. Section 3 defines the primitive. Section 4 specifies CIP. Section 5 is the congestion controller. Section 6 is the runtime, pin pool, mesh, and engines. Section 7 reports the paper suite. Section 8 discusses. Section 9 states limitations. Section 10 concludes.

2 Related work

2.1 Occupancy serving

Orca [2] made continuous batching the default: the scheduler keeps the GPU full by admitting new sequences as others finish. vLLM [1] paged the KV cache so fragmentation would not kill occupancy. Sarathi-Serve [3] splits prefill chunks to hide bubbles. FastServe [4] preempts long generations for shorter ones. FlashAttention [10] cut attention IO. TPU serving [11] scaled the kernel.

All of these systems assume that if a sequence is admitted, decode should proceed as fast as the device allows. The client is a sink. The network is a pipe. Credit, if it exists, is a byte window on the socket that never reaches the residual stream. DART reuses the kernels. It changes the admission question from “is there a free batch slot?” to “is there an outstanding Interest?”

2.2 Disaggregation and KV movement

DistServe [5] and Splitwise [6] split prefill from decode so the two phases do not contend. Mooncake [7] treats KV as the cluster’s first-class resource. CacheGen/LMCache [8] compresses and streams KV. Llumnix [9] reschedules live requests across instances. NIXL-style RDMA movers copy pages between GPUs.

DART reuses those movers as KVConnector backends (memory, file, lmcache, nixl). The difference is the trigger. Handover is not a control plane rescheduling an HTTP request. It is an Interest that arrived with a new locator, or a router that chose a cheaper node because the pin holder released. The continuation object—lease plus kv_root—survives. The HTTP request does not have to.

2.3 QoE pause

Andes [12] is the important comparison, not a strawman. Andes measures that humans read slower than GPUs generate. It pauses generation when a client-side watermark is full, and resumes when the reader drains. That is a coupling between display and kernel.

It is still generate-then-buffer. Tokens sitting in the watermark have already paid residual-stream steps and KV pages. If that pause captured $\geq 90\%$ of IDD’s generated-token win versus push, CIP would be an Andes patch (kill criterion 1, Section 3.3). It does not (Section 7.2): Andes captures 68% of the win and holds 32 unread tokens. IDD inventory high-water is 0. The kernel did not run.

A second difference is the cache unit. Andes pauses a request. DART names a segment. A second process with FileCAS can answer the same name with no GPU (Section 7.4). That is not an Andes feature.

2.4 Named data, media, and structured decode

NDN [17] named immutable objects and pulled them with Interests. An NDN Interest is not a GPU capability: it does not authorize a residual-stream step, cap a token window, or pin KV. DART borrows the pull and the immutability. It adds a lease so a client cannot Interest for 10^6 tokens on someone else’s GPU.

MOQT [14] maps live media onto QUIC objects. A future profile can map CIP Data onto MOQ for fan-out. It must not become the decode authorizer. Byte windows on QUIC are not token credit.

SGLang [15] and XGrammar [16] jump structured spans inside one process: a JSON value or a tool-call argument is produced as a forced chunk rather than token-at-a-time logit masking. DART names that span (`grammar/span/<s>`) so a peer can cache it as one Data object. The kernel ratio on the `tool-args` span is 36 masked launches versus 1 jump (Section 7.3).

Table 1: CIP object kinds. Immutable Data is keyed by $(h_{\text{model}}, \text{kv_root}, \text{kind}, \text{index})$ and never overwritten.

Kind	Name suffix	Authorizes
tokens	tokens/seg/ <i>i</i>	up to W residual-stream steps
grammar	grammar/span/ <i>s</i>	one jump-forward span as one Data
kv	kv/layer/ ℓ /blk/ <i>b</i>	named KV extent fetch
draft	draft/k/ <i>n</i>	speculative prefix (not named in CAS in v1)

2.5 What we refuse to treat as a credit gate

Three designs look like backpressure and are not:

- TCP / HTTP/2 byte windows. They stop the socket, not the kernel.
- max_tokens abort and restart. They re-enter engine admission, race async abort, and lose KV.
- Client-side buffering with a watermark pause (Andes). They still generate into the buffer.

IDD is refuse-to-decode: $W=0$ does not enter the batch.

3 Interest-Driven Decode

3.1 A generation is an address space

A live generation is not an HTTP request bound to a process. It is a family of immutable objects, addressed by Equation 1 and the related kinds in Table 1. The kv_root in the name is the prefix *before* the segment. The Data body carries the child root. Historical names stay valid: committing segment i does not rewrite segment $i-1$.

The mutable cursor is a signed ContinuationLease (Section 4.2). It is a capability, not a cache key. Any compatible producer that can fetch the named KV extents may resume.

3.2 Interest as a compute capability

Definition 1 (Interest). An Interest is a tuple $I = (n, W, T, \ell, L, k)$ where n is a CIP name, $W \in [1, 4096]$ is a token window, T is InterestLifetime, ℓ is a locator, L is an HMAC lease, and k is an InterestKind.

Definition 2 (IDD invariant). Let c be a live continuation. A call engine.decode(c, n) with $n > 0$ is legal at time t only if c has either (i) granted credits > 0 , or (ii) an outstanding grammar span. A CAS hit is not a decode.

Corollary: zero outstanding Interests and no in-flight credits implies the scheduler must not call the engine. The implementation increments decode_steps_skipped, sleeps KV extents off GPU while the pin lease holds, and waits.

Definition 3 (Data). Data is an immutable response: the name, token ids and text, child kv_root, previous kv_root, position, sampler hash, producer id, cache_hit, and stopped.

On a CAS hit the producer returns Data and does not call the engine. Otherwise it grants $\min(W, \text{cwnd})$ credits and may decode at most that many tokens. On lifetime expiry it stops, pins KV, and discards speculative draft. A Nack names the refusal (Table 2).

Disconnect zeroes credits and NACKs pending Interests. That is SSE backpressure that actually stops the kernel.

3.3 Kill criteria

Four criteria decide whether IDD is a product or a patch. They are still in force.

1. **Andes-complete.** If a watermark pause on the same producer captures $\geq 90\%$ of IDD’s generated-token win versus push, *and* Andes inventory stays at the watermark, ship that patch and stop treating CIP as the primitive.

Table 2: Nack reasons. Intermediate nodes may aggregate Interests or refuse. They may not mint decode credit.

Reason	When
no_credit	congestion window / lease has no room
no_model	fingerprint mismatch, or decode failed
busy	continuation handed over; try the new locator
expired	InterestLifetime or lease TTL elapsed
amplification	$W > W_{\max}^{\text{lease}}$ or zero room with no in-flight
unknown_name	CAS miss on a cache-only peer, or no KV blob for handover
done	continuation finished or closed

2. **Batch collapse.** If credit gating drops GPU utilization so far that consumed tok/J gets worse, enable fill-from-other-demand (prefill, embeddings, offline batch) before declaring death. The synthetic single-node runtime sleeps instead; that is correct for the kill-test and incomplete for a packed cluster.
3. **Name tax.** Segments stay 8–32 tokens. Per-token names are forbidden.
4. **Prior art hit.** A system that already gates the *decode kernel* on named consumer Interests.

Criterion 1 is measured in Section 7.2. Criterion 2 is not claimed: this suite has no joule counter on a packed GPU fleet. Criterion 3 is enforced by `segment_size` $\in [1, 128]$ with paper runs at 16, and by Interest coalescing on the same name. Criterion 4 is the related-work claim: we have not found that system.

3.4 Who issues Interests

Credit comes from the consumer, not the GPU.

- DrainPacer — API drainer; W as fast as the caller pulls.
- ReadingPacer($r=30$) — compositor; token bucket at reading speed.
- TtsPacer — mouth as credit source; `realtime_factor=1` is wall-clock speech.
- JsonNeedPacer — pull a span, pause (tool args / JSON value).

OpenAI clients keep working: `POST /v1/chat/completions` with `X-Dart-Pace` / `X-Dart-Window`. Without pace, the facade uses drain credit as SSE is pulled—already stronger than stock vLLM.

4 Continuation Interest Protocol

v1 is JSON over HTTP and WebSocket. It is named computation, not classic NDN: an Interest for a missing segment is a capability to run `decode(kv_root, pos, n)`.

4.1 Names

A CIP name parses as `/cip/<h>/<r>/<rest>`, where h and r are hex and `rest` matches one of Table 1. `model_hash` is `ModelConfig.fingerprint()`: model id, tokenizer hash, RoPE θ , dtype, block size, layers, KV heads, head dim, vocab. A producer that cannot match the fingerprint Nacks `no_model`.

`kv_root` is a SHA-256 Merkle root over layer-block extent digests, sorted by (layer, block_id). This is identity, not a ZK proof. Any compatible producer that can fetch the named extents can resume. The root changes every committed segment. Old Data names stay bound to the old root.

Bytes per extent on the synthetic producer (and the default `ModelConfig`) are

$$2 \times n_{\text{kv_heads}} \times d_{\text{head}} \times B \times \text{nbytes}(\text{dtype}), \quad (3)$$

with $n_{\text{kv_heads}}=8$, $d_{\text{head}}=64$, $B=16$, dtype f16: 32 KiB per layer-block, eight layers.

Algorithm 1 HandleInterest(I): CAS, credit, wait for Data or Nack.**Require:** signed lease L on I ; secret s

```

1:  $c \leftarrow \text{verify}(L, s)$ ;  $c.\text{assertWindow}(I.W)$ 
2:  $C \leftarrow \text{continuation of } c.\text{cont\_id}$ 
3: if  $C$  closed or handed over then
4:   return CAS( $I.n$ ) or Nack(busy/done)
5: end if
6: if CAS contains  $I.n$  then
7:   ACK( $|Data|$ ); return Data with cache_hit
8: end if
9: if  $I.n$  model hash  $\neq C$  fingerprint then
10:  return Nack(no_model)
11: end if
12: if  $I.$ locator changed then
13:  rebind  $C.$ producer ▷ Interest-triggered handover
14: end if
15: ACK tokens since last Interest
16:  $g \leftarrow \text{Grant}(I.W, \text{cwnd}, W_{\max}^{\text{lease}})$ 
17: coalesce duplicate waiters on  $I.n$ ; wake scheduler
18: wait up to  $I.T$  for Data
19: if timeout then
20:   $\text{cwnd} \leftarrow \max(1, \text{cwnd}/2)$ ; drop credits; discard draft
21:  return Nack(expired)
22: end if
23: return Data or Nack from waiter

```

4.2 Lease

An Interest is a compute capability. The lease bound to a continuation caps window, decode quota, and lifetime so a client cannot Interest for a million tokens on someone else’s GPU.

Fields: `cont_id`, `model_hash`, `model_id`, `kv_root`, `pos`, $W_{\max}$ (default 128), `decode_quota`, `expiry_unix`, `producer_hint`, `startup_credit` (default 16). The wire token is `b64url(canonicalJSON) · b64url(HMAC-SHA256)`. Verification uses `compare_digest`. `window > Wmax` is `AmplificationError`. Cache hits are free; decode spends quota. Duplicate Interests for the same name coalesce: no extra credit.

Billing unit, when a fleet chooses to meter: joules and KV-bytes per *consumed* token, plus a prefill fee. Generated-but-unread is the quantity IDD drives to zero.

4.3 Messages and surfaces

Interest, Data, and Nack are Section 3. WebSocket frames add open, ack, and close. HTTP:

Method	Path	Meaning
POST	<code>/v1/continuations</code>	prefill; return capability
POST	<code>.../{id}/interest</code>	pull next object
POST	<code>.../{id}/ack</code>	explicit consume ACK
DELETE	<code>.../{id}</code>	zero credit, sleep KV
POST	<code>/v1/mesh/interest</code>	router: CAS → pin → handover
POST	<code>/v1/handover</code>	adopt named KV (no prefill)
GET	<code>/v1/kv?root=</code>	pin / connector lookup
WS	<code>/v1/cip</code>	framed Interest / Data / Nack
POST	<code>/v1/chat/completions</code>	OpenAI facade

Headers `X-Dart-Pace` and `X-Dart-Window` opt a stock OpenAI client into a reading pacer. Client disconnect cancels the generator and *closes the continuation*.

Algorithm 1 is the producer-side Interest handler as implemented.

Table 3: Congestion signals and the knobs they turn. v1 is AIMD because the ACK is a discrete token count.

Signal	Knob
outstanding Interests / RTT	speculative K (signal; not uncredited decode)
repeated grammar nonterminal	jump-forward span as one Data
InterestLifetime expiry	stop decode, pin KV, discard draft
Interest from a new locator	handover
zero Interests	zero decode
next Interest	ACK + slow-start / AIMD

5 Congestion control is the scheduler

Engine knobs are not YAML hyperparameters. Transport signals map onto inference knobs (Table 3).

5.1 State and grant

Per continuation: $cwnd$ (tokens believed in flight), credits (authorized, not yet produced), in-flight (produced, waiting ACK), $ssthresh$, RTT EWMA, K . Invariant: Equation 2.

On Interest, RTT is updated as $0.8 \text{ RTT} + 0.2 \text{ sample}$ when a sample exists. Room and grant:

$$\text{room} = \max(0, \min(cwnd, W_{\max}^{\text{lease}}, W_{\max}) - \text{in-flight} - \text{credits}), \quad (4)$$

$$g = \min(W, \text{room}). \quad (5)$$

Credits increase by g . Duplicate in-flight credit is not added twice.

`take_decode( $n$ )` returns committed $\min(n, \text{credits})$ and a speculative extra $\min(K, n - \text{committed})$. The extra is counted as `tokens_drafted`. It is not passed to the engine in v1.

5.2 Slow-start, AIMD, timeout

$cwnd$ starts at $W_{\text{init}}=16$ so the first Interest can hide TTFT (`startup_credit` on the lease). The next Interest ACKs the previous segment.

$$cwnd \leftarrow \begin{cases} \min(W_{\max}, cwnd + \text{consumed}) & cwnd < ssthresh, \\ \min(W_{\max}, cwnd + \text{consumed}/cwnd) & \text{otherwise.} \end{cases} \quad (6)$$

On timeout: $ssthresh \leftarrow \max(2, cwnd/2)$, $cwnd \leftarrow \max(1, cwnd/2)$, leftover credits dropped, draft discarded, KV pinned. This is CUBIC/BBR-shaped in intent [13]. v1 is AIMD because the signal is cheap and discrete. `on_ack` / `on_timeout` can be replaced without touching the engine.

5.3 Speculative K

$$K = \text{clamp}(0, \text{round}(cwnd \cdot \text{RTT}/t_{\text{decode}} - cwnd), K_{\max}). \quad (7)$$

K is computed and exported on every continuation (TCP-style prefetch signal). v1 does *not* run uncredited residual-stream steps: a token with no consumer credit is wasted energy and KV, and HTTP vLLM/llama.cpp adapters cannot roll back a draft. When a producer supports verified rollback (native KV snapshot), draft may run ahead of W and must be discarded on expiry without being named in CAS. Until then, `tokens_drafted` counts the would-have prefetch, not GPU work.

5.4 Packing and occupancy

Credit-ready continuations pack up to `max_num_seqs`. $W=0$ does not enter the batch. If the ready set is too small, DART does not mint unread tokens to fill the GPU. A cluster should fill with other paid work (prefill, embeddings, offline batch). The synthetic/single-node runtime sleeps instead—correct for the kill-test, incomplete for a packed cluster, and not swept under the rug (criterion 2).

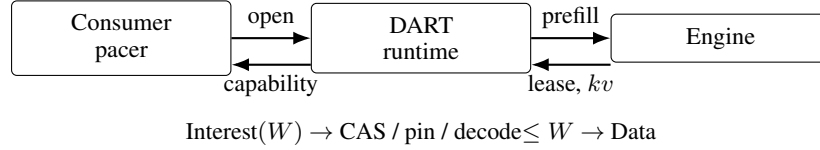


Figure 1: Prefill once. Subsequent work is credited Interests against a named continuation. Disconnect zeroes credit.

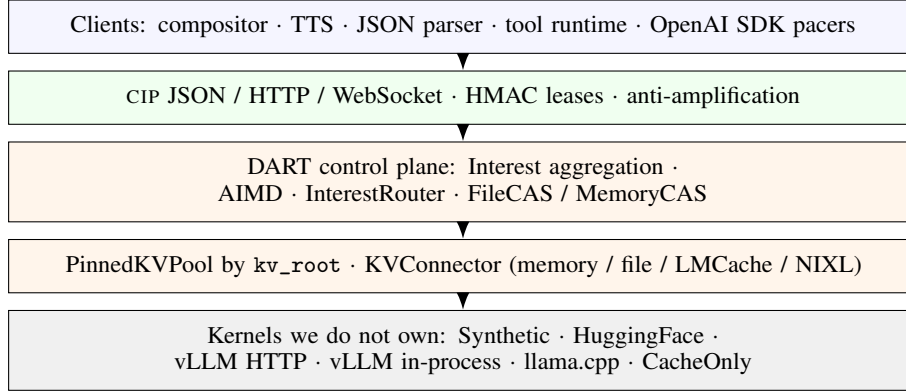


Figure 2: Layer cake. Green/orange is the control plane. Grey is the residual stream. DART never ships a CUDA kernel.

Grammar is a CC signal, not “need N tokens.” Repeated Interest for a nonterminal means “need the next typed span.” The scheduler sets `grammar_span`; the producer may jump-forward. The span is one cacheable Data; its token length still counts against quota and ACK.

6 Architecture

Figure 1 is the implemented loop. Figure 2 is what DART owns versus what it reuses.

6.1 Runtime

`DartRuntime` is one asyncio scheduler, one producer, in-process CAS by default. `open` prefills, signs a lease, pins the initial `kv_root`, and returns a handle. The scheduler loop (Algorithm 2) ticks on Interest wakeup or `poll_interval`.

`consume` is the SDK helper: the pacer’s `next_window` becomes W , names advance with the child root, and EXPIRED retries (cap 8) so a slow engine is not a hard stop. Grammar pacers name `grammar/span/next-value` instead of a token segment.

6.2 Pin, connector, mesh

`PinnedKVPool` indexes engine state by `kv_root`: token ids, sampler, extents, pos, segment index, holder, pin lease. Sleep sets `on_gpu=false` and does not drop the record. Expiry drops it. Eviction prefers sleeping, then oldest `last_access`, and never the newest pin. `adopt` transfers the holder and increments `prefills_skipped` on the destination runtime: resume is installing named KV, not thawing a process.

Connectors expose `put/get/transfer`:

Kind	Persistence	RDMA required
memory	process	no
file	directory of JSON blobs	no
lmcache	same API; real library if installed	no for tests
nixl	accounts bytes/hops; nixl-rdma if importable else memcpy	no for tests

Algorithm 2 SchedulerTick: skip when $W=0$; decode at most granted credits.

```

1: for each live continuation  $C$  do
2:   if  $C$  stopped or over quota then
3:     finish; Nack(done); continue
4:   end if
5:   if lifetime expired and credits  $> 0$  and no waiter then
6:     timeout; discard draft
7:   end if
8:   if credits  $> 0$  or grammar or pending Interest then
9:     wake KV / pin; add  $C$  to ready
10:  else
11:    skipped++; pause(keep) or sleep pin; do not call decode
12:  end if
13: end for
14: batch  $\leftarrow$  ready[: max_num_seqs]
15: for  $C$  in batch do
16:    $(n, K_{\text{extra}}) \leftarrow \text{takeDecode}(\min(\text{segment}, \text{remaining}))$ 
17:   count  $K_{\text{extra}}$  as drafted; call engine.decode( $C, n$ ) only
18:   commit tokens; CAS.put(name); publish new kv_root; complete waiters
19: end for

```

Algorithm 3 RouteInterest(I): CAS, then pin holder, then cheapest plus handover.

```

1: verify lease; lock on  $I$ 's kv_root
2: if CAS has  $I.n$  then
3:   return Data, kind=cas
4: end if
5: if a live node holds the pin then
6:   return that node's HANDLEINTEREST, kind=pin
7: end if
8: if a live continuation matches (cont_id, kv_root) then
9:   return that node, kind=local
10: end if
11:  $d \leftarrow$  cheapest node; pull KV blob;  $d$ .adopt; decode
12: return Data, kind=handover, prefill_skipped

```

A connector miss is a miss. Decode already succeeded if put fails; the runtime logs and continues. Handover without a blob is Nack unknown_name, never a silent re-prefill.

InterestRouter (Algorithm 3) is not sticky to the prefill machine. Cheapest is $\text{cost} \times (1 + \text{awake seqs}) + \text{gpu_bytes}/10^9$. A pin holder is preferred even when it is expensive: moving KV to a cheaper GPU is a handover, not the default. Per-kv_root asyncio lock: two concurrent Interests cannot split-brain adopt.

Handover steps: (1) release_for_handover on the old node (pending Interests Nack busy; CAS still answers); (2) transfer named KV (NIXL-shaped); (3) dest adopt; (4) dest satisfies the Interest. `dart mesh -nodes 3` is the local form.

6.3 Engines

DART never owns the residual stream. Table 4 is the adapter set.

HTTP adapters implement the IDD invariant without forking vLLM: they simply do not POST when $W=0$. They re-enter the engine's request admission path on every credited Interest (Section 9). The in-process plugin is the tighter binding: CreditGatedScheduler admits once, pins blocks, and `schedule_decode(n)` moves `waiting` $\rightarrow$ `running` $\rightarrow$ `waiting`. Memory pressure may evict unpinned prefix-cache victims, never a waiting pin. `DART_VLLM_INPROCESS=1` plus the `vllm` package binds that scheduler to a live GPU process; tests use the synthetic residual stream in the same slots.

The kill-test measures the engine's `kernel_launched` flag. Empty output with that flag false is required when $n \leq 0$.

Table 4: Engine adapters. Paper tables use the *engine* kernel-launch counter, not only wrapper ticks.

Flag	Class	Role
synthetic	SyntheticEngine	Hash tokens, real extents; kill-test default
hf	HuggingFaceEngine	In-process weights; demo SmolLM2-135M
vllm	VLLMChatEngine	HTTP; each Interest is $\text{max_tokens}=W$
vllm-inprocess	InProcessVLLMEngine	Admit once; waiting + pinned blocks
llamacpp	LlamaCppEngine	HTTP $n_predict=W$, cache_prompt
cache	CacheOnlyEngine	FileCAS peer; never decodes

6.4 Implementation notes

The repository is a Python 3.11 package (`src/dart/`): `core` (runtime, leases, CC), `cip` (names, Merkle), `kv` (CAS, pin, connectors), `mesh`, `engine`, `api` (FastAPI + demo compositor), `client` (SDK, pacers), `eval` (paper suite). Public imports stay stable: `from dart import DartRuntime, Interest, ReadingPacer`. Metrics: Prometheus `/metrics` and JSON `/v1/metrics`, including `dart_decode_steps_skipped`, pin adopts, and engine launches. GET `/v1/engine` compares local HTTP POSTs with the engine process forward counter.

7 Evaluation

All numbers in this section are from `dart experiment -suite paper (-seconds 1.0 -max-tokens 96)` unless noted. The producer is `SyntheticEngine` (eight layers, GQA-shaped extents, 32 KiB per layer-block, hash-based tokens). That is a control-plane measurement with real KV accounting, not a GPU-fleet result. Paper tables report *engine* kernel launches (successful generate/completion), not only wrapper ticks. Prompt is fixed: “Write a long essay about named continuations.”

Survive (kill-test): $\text{generated} < 0.9 \times \text{push}$ or lower KV high-water, with $\text{consumption} > 0$.

7.1 Kill test: push versus credit

Table 5 is the inversion test.

Push uses `DrainPacer` with a large window: the kernel runs until the 96-token cap. Six launches of 16-token segments fill the quota. KV high-water is 1 835 008 B (1.75 MiB): prompt plus 96 outputs span seven 16-token blocks times eight layers times 32 KiB.

IDD at 30 tok/s generates 46 tokens and consumes 46. Unconsumed is 0. Generated-token cut versus push is $50/96 = 52\%$. KV high-water cut is $(1.75 - 1.00)/1.75 = 43\%$ (four blocks instead of seven). It launches *more* kernels (31 vs. 6) because each Interest is a short W , not one long push. The engine still only runs when credited; launch count is not the objective. Ninety scheduler ticks skip decode. The kill test survives.

API at 200 tok/s can absorb the cap in one second, so generated tokens match push. Launches rise (81) because the pacer still segments. Skipped ticks drop to 32. This arm exists to show that IDD is not “always generate less”: if the consumer can take 96 tokens, the producer may emit 96. The gate follows demand.

JSON bursty (`JsonNeedPacer`, burst 64, pause 50 ms) generates 21 tokens, 21 launches, 93 skips, 0.75 MiB KV. A parser that pauses between values should not pay for an essay.

7.2 Andes-complete

Andes is reimplemented against the same synthetic producer: generate into a buffer, pause while $|\text{buffer}| \geq 32$, consume at 30 tok/s. It generates 62 tokens (35% cut vs. push) and holds 32 unread tokens at high-water. Once the buffer sits at the watermark, further launches are 1-token top-ups as the consumer drains—hence 32 engine launches, not four large chunks.

IDD generates 46 (52% cut vs. push, 26% cut vs. Andes). Inventory high-water is 0: there is no unread buffer, because those tokens were not encoded.

Andes’ win over push is $96 - 62 = 34$ tokens. IDD’s win is $96 - 46 = 50$. Capture is $34/50 = 0.68$. The Andes-complete kill requires capture ≥ 0.90 and Andes inventory at the watermark. The second clause is true; the first is not. The kill does *not* fire. CIP is not an Andes patch.

Table 5: Push drain versus credit-gated pacers. Unconsumed is generated minus consumed. KV high-water is named extent bytes. Skipped ticks are scheduler loops with $W=0$.

Arm	Gen.	Cons.	Eng. launches	Skipped	KV HW
Push (drain)	96	96	6	0	1.75 MiB
IDD 30 tok/s	46	46	31	90	1.00 MiB
API 200 tok/s	96	96	81	32	1.75 MiB
JSON bursty	21	21	21	93	0.75 MiB

Table 6: Andes watermark pause versus IDD refuse-to-decode. Capture is Andes’ generated-token win over push divided by IDD’s. Kill if capture $\geq 90\%$ and Andes inventory stays at the watermark.

Arm	Generated	Eng. launches	Inventory HW	Capture of IDD
Push	96	6	0	—
Andes ($W=32$)	62	32	32	0.68
IDD 30 tok/s	46	31	0	1.00

The qualitative tell is inventory, not the 16-token gap alone. Andes can be tuned (smaller watermark, slower consume) to generate less. It cannot make unread inventory zero without refusing to decode—at which point it has become IDD.

7.3 Grammar jump-forward

Table 7: the span `{"name": "search", "query": "dart idd"}` is one Interest. Jump-forward tokenizes it as three ids in one kernel and stores one Data. Masking walks the span a token at a time: 36 launches, ratio 36. SGLang/XGrammar already jump inside one process [15, 16]. The DART claim is the name: a peer can satisfy the same span from CAS.

7.4 CAS peer

Process A decodes into FileCAS. Process B starts with CacheOnlyEngine on the same directory and `satisfy_named` the CIP name.

Result: `peer_cache_hit=true`, `producer launches = 1`, `peer launches = 0`, `texts match`, `peer_gpu=false`.

That is the not-Andes defense. The continuation object is the cache unit. A GPU-less peer is a valid producer for a name that already exists. Per-token HTTP replay cannot do this without re-running the kernel or sharing a process.

7.5 Mesh handover

Three in-process nodes, NIXL-shaped connector, costs (10, 5, 1) so node-2 is cheapest. Open on node-0 (one prefill). Table 8 is the route vector.

Assertions that all hold: CAS text equals the first Data; pin route stays on node-0 even though node-2’s score is lower; handover reports `prefill_skipped`, `transfer_bytes=524288`, `destination prefills=0`, and a positive token count. Changing machines did not live-migrate a request.

7.6 Waiting plugin versus HTTP admission

Idle $W=0$ after open: 1 admission, 1 prefill forward, 0 decode forwards, 1 request in waiting, 8 pinned blocks (256 KiB), 0 evictions.

Two Interests: 2 decode forwards, still 1 admission, 0 re-entries, still 8 pinned blocks, request returns to waiting.

Memory pressure: an unpinned prefix-cache victim (4 blocks) is evicted; the waiting pin survives.

HTTP fake-vLLM on the same two Interests: 2 generate POSTs (admission twice).

The plugin tightens KV residency and admission. It does not invent credit-gated decode—the HTTP path already refused to POST at $W=0$. It avoids the re-entry the HTTP path still pays.

Table 7: Typed Interest for grammar/span/tool-args. Jump-forward emits one named Data. Logit masking pays one kernel per constrained token.

Arm	Kernel launches	Tokens (ids)	Text bytes
Logit masking	36	36	36
Jump-forward	1	3	36

Table 8: Three-node mesh. Kernel vector is (node-0, node-1, node-2). Prefills stay (1, 0, 0) after handover: node-2 adopted.

Step	Kind	Node	Kernels after	Note
Interest seg/0	pin	node-0	(1, 0, 0)	first decode on holder
Repeat name	cas	CAS	(1, 0, 0)	no extra kernel
Interest seg/1	pin	node-0	(2, 0, 0)	holder beats cheaper node-2
After release, seg/2	handover	node-2	(2, 0, 1)	512 KiB transfer, skip prefill

8 Discussion

Control plane versus kitchen. A GPU fleet is the kitchen: occupancy batching, paged KV, disaggregation, RDMA. DART is the waiter: it will not send food the table has not ordered. Building a waiter is not building a kitchen, and measuring the waiter on a synthetic producer is not a cluster tok/s number. The paper suite is designed so a reviewer can rerun `dart experiment -suite paper` and get the same cuts.

More launches, less work. IDD 30 tok/s launches 31 kernels versus push’s 6. That is expected. Each kernel is a short credited window. The objective is generated tokens, unread inventory, and KV high-water—not launch count. A production in-process plugin can coalesce ready continuations into one batched forward; HTTP adapters cannot without re-entering admission.

Demand following. API 200 tok/s matches push on tokens. JSON bursty does not. The same invariant covers both: follow W , do not invent a second policy for “fast clients.”

Why names. Pin and CAS are not optional decorations on a credit gate. Without names, a second process cannot answer, handover is live-migration, and grammar jump is trapped in one engine. The Merkle root is deliberately not a proof; it is a lookup key.

Fill-from-other-demand. If a packed cluster idles SM cycles because the credit-ready set is small, those cycles should go to prefill, embeddings, or offline batch—not to unread decode. We do not measure tok/J here (criterion 2). Declaring energy victory from Table 5 would be a category error.

9 Limitations

DART is not a finished GPU fleet that replaces vLLM. HTTP `VLLMChatEngine` / `LlamaCppEngine` re-enter admission; a load spike can 503 a live continuation; prefix cache may evict and force re-prefill. IDD still did not decode without credit; it may pay a re-prefill pinned blocks would have avoided. Measure `GET /v1/engine`: local POSTs versus the engine process forward counter.

Real NIXL RDMA and LMCACHE GPU pages are optional. Tests use `memcpy`. Metrics report `rdma_available=false` when the library is absent. `CreditGatedScheduler` binds to a live GPU vLLM process only when the vllm package is present and `DART_VLLM_INPROCESS=1`. The waiting-plugin numbers in Section 7 use the in-process synthetic residual stream.

The Hugging Face path (SmolLM2-135M-Instruct) is a real-model demo of the same CIP, not the paper-suite producer. We do not mix those numbers into Tables 5–8.

Speculative K is a signal. Uncredited draft is not executed. A producer with verified rollback is future kernel work, not a hidden flag.

If credit gating dropped consumed tok/J because the GPU went idle, fill-from-other-demand must be enabled before declaring death (criterion 2). Per-token names are forbidden; segments stay 8–32 tokens (criterion 3).

We do not claim an IETF standard, an NDN deployment, or MOQ interoperability.

10 Conclusion

Serving should not encode segments nobody has credit to consume. DART makes that a protocol: named continuations, token windows, CAS then pin then decode, handover as the next Interest from a new locator. The kernels stay the kernels. The scheduler is a congestion controller. The paper suite is in the repository; the Andes-complete kill did not fire; the HTTP honesty clause remains.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022.
- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [4] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. FastServe: Fast Distributed Inference Serving for Large Language Models. In *arXiv preprint arXiv:2305.05920*, 2023.
- [5] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [6] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.
- [7] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [8] Yuhua Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.
- [9] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [10] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*, 2022.
- [11] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently Scaling Transformer Inference. In *Proceedings of Machine Learning and Systems*, 2023.
- [12] Jiachen Liu, Zhiyu Lu, Shuowei Shi, Alexander Putta, Peter Wang, Lymeng Chen, Ang Chen, Fan Zhang, Yiping Cheng, Yifan Du, et al. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. *arXiv preprint arXiv:2404.16283*, 2024.
- [13] Van Jacobson. Congestion Avoidance and Control. *ACM SIGCOMM Computer Communication Review*, 18(4):314–329, 1988.
- [14] Luke Curley, Kirill Pugin, Suhas Nandakumar, Victor Vasiliev, and Ian Swett. Media over QUIC Transport. Internet-Draft draft-ietf-moq-transport, 2024.

- [15] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104*, 2024.
- [16] Yixin Dong, Charlie F. Jiang, Yilong Yang, Yuzhen Lin, Zixuan Zhu, Chen Li, et al. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *arXiv preprint arXiv:2411.15100*, 2024.
- [17] Van Jacobson, Diana K. Smetters, James D. Thornton, Michael F. Plass, Nicholas H. Briggs, and Rebecca L. Braynard. Networking Named Content. In *Proceedings of the 5th International Conference on Emerging Networking Experiments and Technologies*, 2009.

A Paper-suite command

```
dart experiment --suite paper --seconds 1.0 --max-tokens 96
```

Arms: push drain, reading 30 tok/s, API 200 tok/s, JSON bursty, Andes watermark 32, grammar tool-args, FileCAS peer, three-node NIXL-shaped mesh, waiting plugin versus HTTP fake-vLLM. Default synthetic seed is 0 for kill/Andes, 1 for grammar, 3 for CAS, 11 for mesh, 2 for waiting. Install the package in editable mode, then rerun from the repository root.

B Default configuration

Paper runs use RuntimeConfig with `segment_size=16`, `w_init=16` (128 for push), `w_max=128` (512 for push), `startup_credit=16`, `poll_interval_s=0.002`, `interest_lifetime_s=5`, `lease_ttl_s ≥ 60`. Synthetic ModelConfig: 8 layers, 8 KV heads, head dim 64, block size 16, f16, vocab 32 000. Extent bytes = $2 \times 8 \times 64 \times 16 \times 2 = 32$ KiB; KV high-water scales with occupied blocks times layers.

C Module map

`dart.core.runtime` scheduler and Interest handler; `dart.core.cc` AIMD; `dart.core.lease` HMAC capability; `dart.cip.protocol` names and messages; `dart.cip.merkle` kv_root; `dart.kv.pin` pin table; `dart.kv.kvconn` connectors; `dart.mesh.router` CAS/pin/handover; `dart.engine.*` adapters; `dart.eval.experiment` paper_suite; `dart.client.consumers` pacers. Kill criteria and HTTP honesty: repository docs/limitations.md.